DIPLOMADO EN MACHINE LEARNING Y DATA SCIENCE AVANZADO · DEEP LEARNING

Tarea 2 · Redes neuronales con Keras

Clasificación de dígitos y regresión de diabetes con el mismo flujo de trabajo



Guía de estudio: el problema, la teoría de fondo, el código explicado y las decisiones de diseño

Módulo 5, unidad 2 · Universidad Nacional de Colombia, Facultad de Ingeniería

Notebook: `v2_M5U2_Tarea_2_Resuelto.ipynb`

Natalia Patricia Remolina Rodríguez · 7 de noviembre de 2026

✓ Calificación en UNCode: 100 %, 18 de 18 pruebas en 100

La tarea en una página

El problema

- **Parte 1:** reconocer dígitos escritos a mano en imágenes de 8 × 8 píxeles, 10 clases.
- **Parte 2:** predecir un indicador continuo del avance de la diabetes a partir de 10 variables clínicas.

La solución

- **Nueve funciones** que cubren el flujo de Keras: preparar, definir, compilar, configurar *callbacks*, entrenar y evaluar.
- La misma red densa sirve para ambos problemas: solo cambian **la salida y la pérdida**.

Las decisiones clave

- **API funcional** con un ciclo de capas ocultas.
- `ModelCheckpoint` con `save_weights_only` y `EarlyStopping` con `restore_best_weights`.
- La parte 2 **reutiliza** las funciones de la parte 1.

El resultado

- Las 18 pruebas en 100 y envío con **100 %**, a tiempo.
- **92,63 %** de accuracy en dígitos.
- **44,67** de MAE en diabetes, 29 % mejor que predecir el promedio.

CÓMO LEER ESTA GUÍA

La guía avanza de lo general a lo concreto:

- La **parte I** plantea los dos problemas.
- La **parte II** explica la teoría necesaria.
- La **parte III** recorre el código función por función.
- La **parte IV** reúne las decisiones de diseño que no estaban explícitas.
- La **parte V** interpreta los resultados.
- La **parte VI** cierra con preguntas de autoevaluación.

Las cajas de colores tienen siempre el mismo sentido:

QUÉ SE PEDÍA
la instrucción del enunciado.

INTUICIÓN
la idea detrás del código.

INTERVENCIÓN
una decisión que hubo que tomar.

OJO
un detalle fácil de pasar por alto.

Contenido

Parte I · El problema

1. Qué pide la tarea
2. Cómo califica UNCode

Parte II · La teoría de fondo

3. La red densa
4. Clasificación multiclase
5. Regresión
6. Validación y callbacks

Parte III · La solución función por función

7. Preparación
8. Parte 1 · clasificación
9. Entrenamiento más largo
10. Parte 2 · regresión

Parte IV · Las decisiones

11. Lo que no estaba explícito
12. Lecciones

Parte V · Resultados

13. Las pruebas
14. Clasificación
15. Regresión
16. Elegir salida y pérdida

Parte VI · Para consolidar

17. Preguntas
18. Glosario
19. Conclusiones

PARTE I

El problema

Dos problemas distintos resueltos con el mismo flujo de trabajo.

1. Qué pide la tarea

	Parte 1 · Clasificación	Parte 2 · Regresión
Pregunta	¿Qué dígito, de 0 a 9, hay en la imagen?	¿Cuánto avanzará la diabetes en un año?
Datos	*Digits*: 1.797 imágenes de 8 × 8 = 64 píxeles	*Diabetes*: 442 pacientes con 10 variables clínicas normalizadas
División	1.078 de entrenamiento y 719 de prueba, estratificada	309 de entrenamiento y 133 de prueba
Objetivo	Una de 10 clases	Un número entre 25 y 346
Salida de la red	10 neuronas con *softmax*	1 neurona lineal
Pérdida y métrica	Entropía cruzada categórica y accuracy	MAE

Las nueve funciones calificadas

Punto	Función	Qué hace
1.1	`one_hot_labels`	Etiquetas a formato *one-hot*
1.2	`define_model`	Red densa con salida *softmax*
1.3	`compile_model`	Compilar con pérdida, optimizador y métricas
1.4	`define_callbacks`	Guardar el mejor modelo y detener a tiempo
1.5	`train_model`	Entrenar con validación
1.6	`evaluate_model`	Accuracy en prueba
2.1	`define_regressor`	Red densa con salida lineal
2.2	`train_regressor`	Compilar con MAE y entrenar, reutilizando 1.3 a 1.5
2.3	`evaluate_regreesor`	MAE en prueba

2. Cómo califica UNCode

Cada función tiene dos pruebas, 18 en total. El calificador compara formas, número de parámetros, la configuración de compilación y de los *callbacks*, y los valores de pérdida y accuracy. Varias funciones fijan la semilla al inicio, `keras.utils.set_random_seed(0)`, para que los resultados sean reproducibles y comparables.

PARTE II

La teoría de fondo

Lo necesario para entender cada decisión del código.

3. La red densa

Una capa densa conecta cada entrada con cada neurona: cada neurona calcula `z = w · x + b` y aplica una activación. Apilar capas permite aprender relaciones no lineales.

Pieza	Qué hace
ReLU	`max(0, z)` : deja pasar los valores positivos y anula los negativos. Es la activación estándar de las capas ocultas
Parámetros de una capa	entradas × neuronas + neuronas. Por ejemplo, 64 × 16 + 16 = 1.040
API funcional	`Input` → capas aplicadas como funciones → `Model` . Permite construir con un ciclo una red de profundidad variable



Figura 1. La red de prueba de la parte 1. El regresor de la parte 2 tiene la misma forma con 10 entradas, dos capas de 8 y una salida lineal: 169 parámetros.

4. Clasificación multiclase

4.1 Softmax

Convierte las salidas de la última capa en probabilidades positivas que suman 1:

```
softmax(z)_i = e^(z_i) / Σ_j e^(z_j)
```

Ejemplo del enunciado: `z = [5, 3, 4]` da `[0,665, 0,090, 0,245]` . La clase predicha es la 0, la de mayor probabilidad.

4.2 One-hot y entropía cruzada categórica

Para comparar con 10 probabilidades, cada etiqueta se convierte en un vector de 10 posiciones con un solo 1: el dígito 3 es `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]` . La pérdida es `−log` de la probabilidad asignada a la clase correcta:

Probabilidad de la clase correcta	Pérdida
0,665	0,41
0,090	2,41

Equivocarse con confianza cuesta mucho más que acertar con dudas.

5. Regresión

Cuando el objetivo es un número continuo, la salida no puede ser *sigmoid* ni *softmax*, que acotan el rango: se usa una neurona **lineal**. La pérdida mide la distancia entre la predicción y el valor real:

Pérdida	Fórmula	Característica
MAE, error absoluto medio	`mean(|y − ŷ|)`	Se lee en las mismas unidades del objetivo; menos sensible a valores extremos
MSE, error cuadrático medio	`mean((y − ŷ)²)`	Castiga más los errores grandes

La tarea usa MAE: un MAE de 44,67 significa que, en promedio, la predicción se equivoca en unas 45 unidades del indicador.

6. Validación y callbacks

- **Validación:** `validation_split=0.2` aparta el 20 % final del entrenamiento para medir, en cada época, cómo le va al modelo con datos que no usa para ajustar pesos. La prueba queda intacta hasta el final.
- **`ModelCheckpoint`:** guarda los pesos cada vez que la métrica vigilada mejora.
- **`EarlyStopping`:** detiene el entrenamiento si la métrica no mejora durante `patience` épocas, y con `restore_best_weights` vuelve a los pesos de la mejor.

> **INTUICIÓN · POR QUÉ VIGILAR LA VALIDACIÓN**
> La pérdida de entrenamiento casi siempre sigue bajando, incluso cuando el modelo empieza a memorizar. La de validación deja de mejorar cuando aparece el sobreajuste: es la señal para detenerse.

PARTE III

La solución, función por función

El código tal como quedó y qué hace cada parte.

7. Preparación

CÓDIGO · CELDA 10

```
X, y = datasets.load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size = 0.4, stratify=y, random_state = 50)
```

Carga 1.797 imágenes como vectores de 64 píxeles y reserva el 40 % para prueba, con la misma proporción de cada dígito. Antes, la celda de librerías actualizó scikit-learn de 1.6.1 a 1.9.1 y fijó las semillas de NumPy y Keras.

8. Parte 1 · Clasificación

8.1 one_hot_labels

CÓDIGO · CELDA 18

```
def one_hot_labels(y_train, y_test):
    num_classes = int(max(np.max(y_train), np.max(y_test))) + 1
    y_train_ohe = keras.utils.to_categorical(y_train, num_classes=num_classes)
    y_test_ohe = keras.utils.to_categorical(y_test, num_classes=num_classes)
    return y_train_ohe, y_test_ohe
```

Parte del código	Qué hace
`max(np.max(y_train), np.max(y_test)) + 1`	Número de clases a partir de ambos conjuntos
`to_categorical(..., num_classes)`	Convierte cada etiqueta en su vector *one-hot* con el mismo ancho en ambos

8.2 define_model

CÓDIGO · CELDA 25

```
def define_model(input_shape, hidden_layers, hidden_units, output_units):
    keras.utils.set_random_seed(0)
    inputs = keras.layers.Input(shape=(input_shape,))
    x = inputs
    for _ in range(hidden_layers):
        x = keras.layers.Dense(hidden_units, activation='relu')(x)
    outputs = keras.layers.Dense(output_units, activation='softmax')(x)
    model = keras.Model(inputs=inputs, outputs=outputs)
    return model
```

Parte del código	Qué hace

`Input(shape=(input_shape,))`	Entrada de 64 píxeles
`for _ in range(hidden_layers)`	Crea tantas capas ocultas como se pidan; cada una se conecta a la anterior
`Dense(hidden_units, activation='relu')(x)`	Capa oculta con ReLU
`Dense(output_units, activation='softmax')(x)`	Una probabilidad por clase
`keras.Model(inputs, outputs)`	Registra el grafo; es la API funcional que exige el enunciado

8.3 compile_model

CÓDIGO · CELDA 32

```
def compile_model(model, loss, optimizer, metrics):
    keras.utils.set_random_seed(0)
    model.compile(loss=loss, optimizer=optimizer, metrics=metrics)
    return model
```

Asocia la pérdida, el optimizador y las métricas recibidos y devuelve el mismo modelo, listo para entrenar.

8.4 define_callbacks

CÓDIGO · CELDA 39

```
def define_callbacks(filepath, metric, mode, patience):
    checkpoint = keras.callbacks.ModelCheckpoint(filepath=filepath, monitor=metric, mode=mode,
                                                 save_best_only=True, save_weights_only=True,
                                                 verbose=1)
    stopping = keras.callbacks.EarlyStopping(monitor=metric, mode=mode, patience=patience,
                                             restore_best_weights=True, verbose=1)
    callbacks = [checkpoint, stopping]
    return callbacks
```

Parte del código	Qué hace
`monitor=metric, mode=mode`	Qué métrica vigilar y si debe bajar, `min`, o subir, `max`
`save_best_only=True`	Solo guarda cuando la métrica mejora
`save_weights_only=True`	Guarda solo los pesos; Keras 3 lo exige con rutas `.weights.h5`
`patience=patience`	Épocas sin mejora antes de detenerse
`restore_best_weights=True`	Al detenerse, vuelve a los pesos de la mejor época

8.5 train_model

CÓDIGO · CELDA 46

```
def train_model(model, X_train, y_ohe_train, epochs, batch_size, validation_rate, callbacks):
    keras.utils.set_random_seed(0)
    history = model.fit(X_train, y_ohe_train, epochs=epochs, batch_size=batch_size,
                        validation_split=validation_rate, callbacks=callbacks)
    return history, model
```

`validation_split` aparta la fracción final del entrenamiento para validar, y `callbacks` activa el checkpoint y el early stopping. Devuelve el registro de métricas y el modelo.

8.6 evaluate_model

CÓDIGO · CELDA 54

```
def evaluate_model(model, X_test, y_ohe_test):
    loss, acc = model.evaluate(X_test, y_ohe_test)
    return acc
```

`evaluate` devuelve la pérdida y la métrica. El desempaquetado en dos valores funciona porque el modelo se compiló con una sola métrica.

SALIDA DE LA CELDA 56 · 5 ÉPOCAS CON TASA 0,001

```
Accuracy en test: 0.4186
```

9. Entrenamiento más largo, no calificable

CÓDIGO · CELDA 62, RESUMIDO

```
final_model = define_model(64, 2, 16, 10)
final_model = compile_model(final_model, keras.losses.categorical_crossentropy,
                            keras.optimizers.Adam(learning_rate=1e-2),
                            metrics=['acc'])
final_checkpoint, final_stopping = define_callbacks('best_model.weights.h5',
                                                    'val_loss', 'min', 10)
history, final_model = train_model(final_model, X_train, y_train_ohe, epochs=50,
                                   batch_size=64, validation_rate=0.2,
                                   callbacks=[final_checkpoint, final_stopping])
```

SALIDA

```
Epoch 30: early stopping
Restoring model weights from the end of the best epoch: 20.
Accuracy en test: 0.9263
```

Con una tasa 10 veces mayor, hasta 50 épocas y el corte vigilando la validación, la accuracy pasa de 41,86 % a **92,63 %**. La sección 14 analiza las curvas.

10. Parte 2 · Regresión

10.1 define_regressor

CÓDIGO · CELDA 72

```
def define_regressor(input_shape, hidden_layers, hidden_units, output_units):
    keras.utils.set_random_seed(0)
    keras.backend.clear_session()
    inputs = keras.layers.Input(shape=(input_shape,))
    x = inputs
    for _ in range(hidden_layers):
        x = keras.layers.Dense(hidden_units, activation='relu')(x)
    outputs = keras.layers.Dense(output_units, activation='linear')(x)
```

```
    model = keras.Model(inputs=inputs, outputs=outputs)
    return model
```

Es `define_model` con una sola diferencia: `activation='linear'` en la salida, que puede devolver cualquier número real.

10.2 train_regressor

CÓDIGO · CELDA 80

```
def train_regressor(model, filepath, X_train, y_train, epochs, batch_size,
                    learning_rate, validation_rate):
    model = compile_model(model, keras.losses.MeanAbsoluteError(),
                          keras.optimizers.Adam(learning_rate=learning_rate), metrics=['mae'])
    callbacks = define_callbacks(filepath, 'val_loss', 'min', 10)
    history, model = train_model(model, X_train, y_train, epochs, batch_size,
                                 validation_rate, callbacks)
    return history, model
```

Parte del código	Qué hace
`compile_model(..., MeanAbsoluteError(), Adam(...), ['mae'])`	Reutiliza 1.3 con MAE como pérdida y métrica
`define_callbacks(filepath, 'val_loss', 'min', 10)`	Reutiliza 1.4 vigilando la validación con paciencia 10
`train_model(...)`	Reutiliza 1.5; las etiquetas son números, no *one-hot*

10.3 evaluate_regreesor

CÓDIGO · CELDA 89

```
def evaluate_regreesor(model, X_test, y_test):
    loss, mae = model.evaluate(X_test, y_test)
    return mae
```

SALIDA DE LA CELDA 91

```
MEA en test: 44.6665
```

El nombre lleva doble «e» y la etiqueta dice MEA: no se corrigieron porque el calificador busca la función con ese nombre exacto.

PARTE IV

Las decisiones de diseño

Todas las pruebas pasaron en el primer intento; estas son las decisiones que lo permitieron.

11. Lo que no estaba explícito

Decisión	Qué decía el enunciado	Qué se hizo y por qué
Número de clases	Usar `to_categorical`	Calcularlo con ambos conjuntos, para que las dos matrices tengan el mismo ancho aunque falte una clase en alguno
Formato del checkpoint	Guardar el mejor modelo	`save_weights_only=True`: en Keras 3, una ruta `.weights.h5` exige guardar solo los pesos; si no, da error
Pesos al terminar	Una nota pide volver a los mejores	`restore_best_weights=True`; sin esto, el modelo queda con los pesos de la última época
Paciencia del regresor	No la fija	10 épocas sobre `val_loss`, como en el entrenamiento largo y en la salida esperada
Reutilizar funciones	«Use las funciones que definió anteriormente»	`train_regressor` llama a 1.3, 1.4 y 1.5 en lugar de repetir código
Nombre con errata	`evaluate_regreesor`	Se conservó: el calificador lo busca así

OJO · UNA SALIDA ESPERADA EQUIVOCADA

Para los datos de regresión, la salida esperada dice `((309, 10), (133, 10), 346.0, 25.0)`, pero el código imprime `y_test_2.min()`, que vale **40.0**: 25 es el mínimo del entrenamiento. La salida del notebook es la correcta.

12. Lecciones

LECCIONES

1. **El flujo de Keras es siempre el mismo:** definir, compilar, entrenar y evaluar.
2. **Escribir funciones reutilizables ahorra código:** la parte 2 se resolvió con 12 líneas.
3. **Los valores por defecto de las librerías cambian entre versiones:** `save_weights_only` es obligatorio en Keras 3 con esa extensión.
4. **Las salidas esperadas también pueden tener errores**; hay que verificarlas contra el código.

PARTE V

Resultados y análisis

Cuánto aprendió cada modelo y frente a qué compararlo.

13. Las pruebas

Punto	Función	Pruebas	Nota
1.1	`one_hot_labels`	1_1_1, 1_1_2	100, 100
1.2	`define_model`	1_2_1, 1_2_2	100, 100
1.3	`compile_model`	1_3_1, 1_3_2	100, 100
1.4	`define_callbacks`	1_4_1, 1_4_2	100, 100
1.5	`train_model`	1_5_1, 1_5_2	100, 100
1.6	`evaluate_model`	1_6_1, 1_6_2	100, 100
2.1	`define_regressor`	2_1_1, 2_1_2	100, 100
2.2	`train_regressor`	2_2_1, 2_2_2	100, 100
2.3	`evaluate_regreesor`	2_3_1, 2_3_2	100, 100
Envío oficial con `submit_task`			**100 %**

14. Clasificación



Figura 2. Pérdida de entrenamiento y de validación en las 30 épocas que corrió el entrenamiento largo.

- **Sobreajuste desde la época 15:** la pérdida de entrenamiento siguió bajando hasta 0,01, mientras la de validación oscilaba entre 0,21 y 0,30 sin mejorar de forma sostenida.

- **El early stopping hizo su trabajo:** la mejor validación, 0,2077, fue en la época 20; en las 10 siguientes no mejoró, así que se detuvo en la 30 y restauró los pesos de la 20.

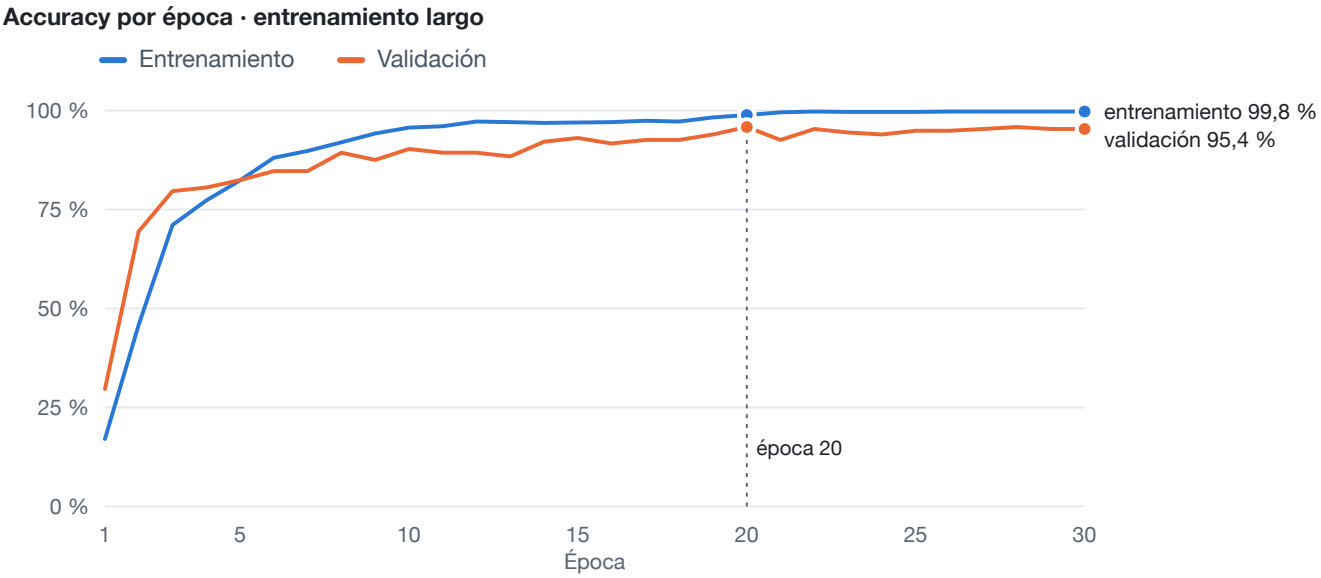


Figura 3. En entrenamiento la accuracy llega casi a 100 %; en validación se estabiliza cerca de 95 %. La distancia entre ambas es el sobreajuste.

Qué dígitos falla el modelo

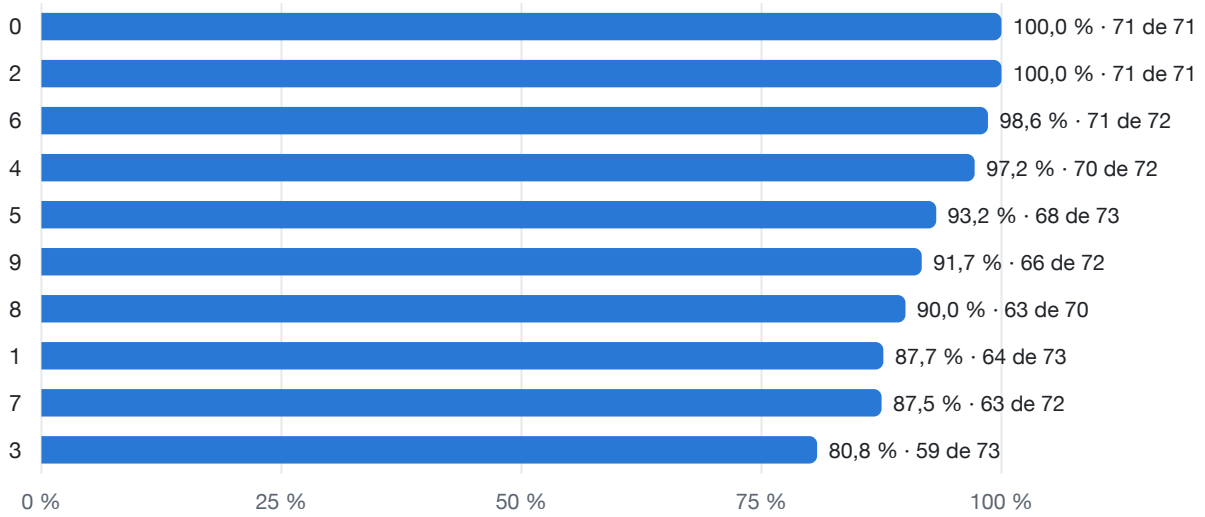


Figura 4. Accuracy por dígito en las 719 imágenes de prueba, de una reproducción aparte del modelo final: el notebook no imprime la matriz de confusión. Los aciertos suman 666, el 92,63 %.

El 3 es el dígito más difícil. Las confusiones más frecuentes son 1 → 8, con 7 casos; 3 → 9, con 6; y 7 → 4, con 5: dígitos con trazos parecidos en solo 8 × 8 píxeles.

15. Regresión

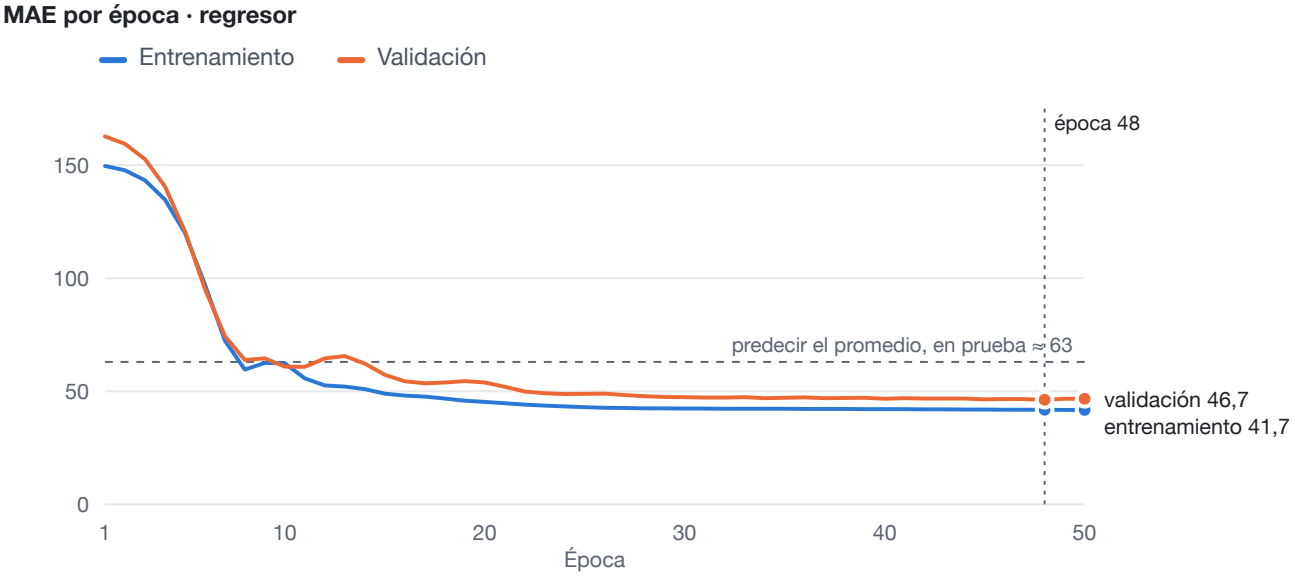


Figura 5. MAE de entrenamiento y de validación en las 50 épocas. La línea de trazos es el MAE en prueba de predecir siempre el promedio.

- **El early stopping no llegó a activarse:** la validación siguió mejorando de a poco, con su mejor valor, 46,28, en la época 48.
- **No hay sobreajuste marcado:** entrenamiento y validación terminan cerca, 41,7 y 46,7.

Modelo	MAE en prueba
Predecir siempre el promedio del entrenamiento	62,98
Red 10 → 8 → 8 → 1 de la tarea	**44,67**
Regresión lineal de scikit-learn, misma partición	43,46

CONCLUSIÓN

La red reduce el error un 29 % frente a predecir el promedio, pero **una regresión lineal simple logra casi lo mismo**. Con 309 muestras y 10 variables tabulares, una red pequeña no tiene mucho más que aprender. Una red neuronal no siempre le gana a un modelo clásico.

16. Cómo elegir la salida y la pérdida

Problema	Salida	Pérdida	Etiquetas	Métrica
Binario	1 neurona `sigmoid`	`BinaryCrossentropy`	0 o 1	accuracy
Multiclase	c neuronas `softmax`	`CategoricalCrossentropy`	*one-hot*	accuracy
Multiclase con enteros	c neuronas `softmax`	`SparseCategoricalCrossentropy`	0 a c – 1	accuracy
Regresión	1 neurona `linear`	MAE o MSE	número real	MAE

PARTE VI

Para consolidar

Preguntas, glosario y conclusiones.

17. Preguntas de autoevaluación

Conviene intentar responder antes de leer la respuesta.

1. ¿Por qué la salida de la parte 1 tiene 10 neuronas con softmax?

Porque hay 10 clases y softmax devuelve una probabilidad por clase que suma 1.

2. ¿Por qué hay que convertir las etiquetas a one-hot?

Porque la entropía cruzada categórica compara un vector de 10 probabilidades con un vector de 10 posiciones.

3. ¿Qué cambia entre `define_model` y `define_regressor`?

Solo la activación de salida: *softmax* para clasificar y lineal para predecir un número.

4. ¿Cuántos parámetros tiene una capa densa de 64 entradas y 16 neuronas?

64 × 16 + 16 = 1.040: un peso por conexión y un sesgo por neurona.

5. ¿Qué pasaría sin `restore_best_weights=True`?

El modelo quedaría con los pesos de la última época, la 30, que están más sobreajustados que los de la 20.

6. ¿Por qué se vigila `val_loss` y no `loss`?

Porque la pérdida de entrenamiento sigue bajando aunque el modelo memorice; la de validación avisa cuando deja de generalizar.

7. ¿Por qué el regresor corrió las 50 épocas?

Porque la validación siguió mejorando de a poco y nunca pasaron 10 épocas seguidas sin mejora.

8. ¿Qué indica que una regresión lineal logre casi el mismo MAE?

Que con pocos datos tabulares la relación es casi lineal y la red no aporta mucho más.

18. Glosario

Término	Definición breve
Capa densa	Capa en la que cada neurona se conecta con todas las entradas
ReLU	Activación `max(0, z)` de las capas ocultas
Softmax	Convierte puntajes en probabilidades que suman 1
One-hot	Vector con un 1 en la posición de la clase y 0 en el resto
Entropía cruzada categórica	Pérdida –log de la probabilidad de la clase correcta
MAE	Error absoluto medio, en las unidades del objetivo
API funcional	Forma de construir modelos aplicando capas como funciones
Validación	Datos apartados del entrenamiento para monitorear el modelo
`ModelCheckpoint`	Guarda los pesos cuando la métrica mejora
`EarlyStopping`	Detiene el entrenamiento cuando la métrica deja de mejorar
Sobreajuste	Mejorar en entrenamiento mientras se empeora en datos nuevos
Línea base	Resultado de una estrategia trivial, para dimensionar el del modelo

19. Conclusiones

- **El problema:** clasificar dígitos y predecir el avance de la diabetes con redes densas en Keras.
- **La solución:** nueve funciones que forman el flujo completo, con la parte 2 construida sobre la parte 1.
- **Las decisiones clave:** API funcional con un ciclo, `save_weights_only` y `restore_best_weights` en los *callbacks*, y vigilar la validación.
- **El resultado:** 18 de 18 pruebas y 100 %. 92,63 % de accuracy en dígitos y 44,67 de MAE en diabetes.
- **Lo que enseña el análisis:** el early stopping evitó quedarse con pesos sobreajustados, y en regresión una red pequeña apenas iguala a una regresión lineal: siempre hay que comparar con una línea base.

Anexo · registro del entrenamiento largo

Época	Pérdida	Accuracy	Pérdida de validación	Accuracy de validación
1	2,5997	0,1705	1,9380	0,2963
2	1,5424	0,4582	0,9903	0,6944
3	0,8278	0,7111	0,6683	0,7963
4	0,6229	0,7726	0,5718	0,8056
5	0,4979	0,8237	0,4662	0,8241
6	0,3674	0,8805	0,4546	0,8472
7	0,3074	0,8979	0,4032	0,8472
8	0,2502	0,9200	0,3349	0,8935
9	0,1804	0,9420	0,3307	0,8750

10	0,1440	0,9571	0,3262	0,9028
11	0,1306	0,9606	0,3140	0,8935
12	0,1107	0,9722	0,3417	0,8935
13	0,0938	0,9710	0,4096	0,8843
14	0,0939	0,9687	0,2917	0,9213
15	0,0956	0,9698	0,2231	0,9306
16	0,0804	0,9710	0,3018	0,9167
17	0,0772	0,9745	0,2641	0,9259
18	0,0693	0,9722	0,2321	0,9259
19	0,0627	0,9826	0,2726	0,9398
20	0,0427	0,9884	0,2076	0,9583
21	0,0316	0,9954	0,2573	0,9259
22	0,0234	0,9977	0,2174	0,9537
23	0,0192	0,9965	0,2362	0,9444
24	0,0186	0,9965	0,2470	0,9398
25	0,0156	0,9965	0,2290	0,9491
26	0,0139	0,9977	0,2464	0,9491
27	0,0128	0,9977	0,2504	0,9537
28	0,0120	0,9977	0,2410	0,9583
29	0,0116	0,9977	0,2518	0,9537
30	0,0104	0,9977	0,2422	0,9537

Créditos del notebook original: profesor Fabio Augusto González, asistentes docentes Juan Sebastián Malagón Torres, Santiago Toledo Cortés y Juan Sebastián Lara, diseño de imágenes de Mario Andrés Rodríguez Triana y coordinador de virtualización Edder Hernández Forero. Universidad Nacional de Colombia, Facultad de Ingeniería.